

# Fichas Técnicas de Estructuras Arbóreas



## DATOS DEL ESTUDIANTE

Facultad: Ingeniería en Sistemas y Ciencias de la Computación  
Curso: Estructuras de Datos  
Estudiante: Roberto Estuardo Chavarría Zelada  
Carnet: 202504060  
Fecha: 10 de mayo de 2026

## OBJETIVO DEL DOCUMENTO

Presentar cinco fichas técnicas profesionales sobre estructuras arbóreas usadas en software: BST, AVL, Árbol B/B+, Trie y Heap. Cada ficha resume definición, complejidad Big O, mecánica interna, ventajas, limitaciones y un caso de uso industrial.

## ESTRUCTURAS INCLUIDAS

**01** **Árbol Binario de Búsqueda (BST)**  
Búsqueda ordenada por comparación.

**02** **Árbol Balanceado (AVL)**  
BST con balance automático por rotaciones.

**03** **Árbol B / B+**  
Árbol multicamino para grandes volúmenes y disco.

**04** **Trie (Árbol de Prefijos)**  
Búsqueda por prefijos y claves tipo texto.

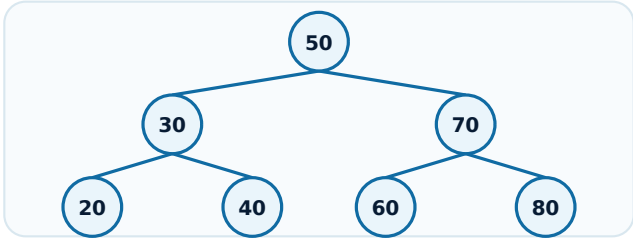
**05** **Heap (Montículo Min/Max)**  
Estructura base para colas de prioridad.

Referencias generales: Cormen, Leiserson, Rivest y Stein (Introduction to Algorithms); Sedgwick y Wayne (Algorithms); documentación de Java Collections; documentación técnica de bases de datos y sistemas de archivos.

# Árbol Binario de Búsqueda (BST)

Estructura de comparación para almacenar claves ordenadas

## IDENTIFICACIÓN Y VISUALIZACIÓN



Definición: árbol binario en el que cada nodo almacena una clave. Los valores menores quedan en el subárbol izquierdo y los mayores en el subárbol derecho. Esta regla permite descartar ramas completas durante una búsqueda.

## ANÁLISIS DE COMPLEJIDAD (BIG O)

| Operación   | Peor caso | Caso promedio | Observación                |
|-------------|-----------|---------------|----------------------------|
| Búsqueda    | $O(n)$    | $O(\log n)$   | Depende de la altura.      |
| Inserción   | $O(n)$    | $O(\log n)$   | Compara hasta hallar hoja. |
| Eliminación | $O(n)$    | $O(\log n)$   | Incluye 0, 1 o 2 hijos.    |

$n$  = cantidad de nodos. El peor caso aparece cuando el árbol se degenera en una lista.

## CARACTERÍSTICAS MECÁNICAS

La organización se mantiene mediante comparaciones: si el nuevo valor es menor que el nodo actual avanza a la izquierda; si es mayor avanza a la derecha. No posee balance automático, por eso su rendimiento depende de la forma final del árbol. El recorrido In-Order imprime las claves en orden ascendente.

### MAYOR VENTAJA

Es simple, directo y muy útil para entender búsquedas ordenadas. Permite insertar, buscar y recorrer datos de manera lógica sin necesidad de ordenar todo el conjunto después de cada operación.

### MAYOR LIMITACIÓN

Si los datos llegan ya ordenados o casi ordenados, la altura puede crecer demasiado. En ese caso el BST pierde eficiencia y se comporta como una lista enlazada.

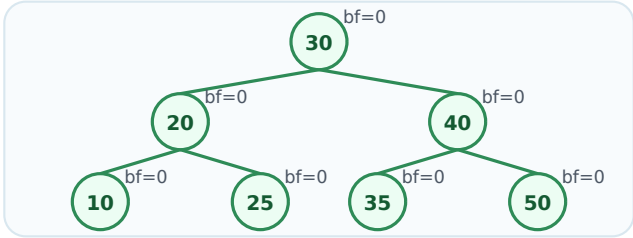
### CASO DE USO EN LA INDUSTRIA

Módulos de catálogo o inventario en memoria donde se guardan códigos de productos y se requiere listarlos en orden. También sirve como base conceptual de mapas y conjuntos ordenados usados en software profesional.

# Árbol Balanceado (AVL)

BST auto-balanceado con altura controlada

## IDENTIFICACIÓN Y VISUALIZACIÓN



Definición: árbol binario de búsqueda que mantiene balanceada su altura. En cada nodo se controla el factor de balance, normalmente entre -1, 0 y 1, para evitar que la estructura se degenera.

## ANÁLISIS DE COMPLEJIDAD (BIG O)

| Operación   | Peor caso   | Caso promedio | Observación                |
|-------------|-------------|---------------|----------------------------|
| Búsqueda    | $O(\log n)$ | $O(\log n)$   | Altura garantizada.        |
| Inserción   | $O(\log n)$ | $O(\log n)$   | Puede requerir rotaciones. |
| Eliminación | $O(\log n)$ | $O(\log n)$   | Rebalancea al subir.       |

$n$  = cantidad de nodos. La altura se mantiene proporcional a  $\log n$ .

## CARACTERÍSTICAS MECÁNICAS

Funciona como un BST, pero después de insertar o eliminar revisa el factor de balance de los nodos afectados. Si la diferencia de alturas rompe la regla, aplica rotaciones simples o dobles: izquierda, derecha, izquierda-derecha o derecha-izquierda. Con esto conserva búsquedas predecibles.

### MAYOR VENTAJA

Ofrece tiempos estables para búsqueda, inserción y eliminación. Es ideal cuando se necesita acceso rápido a datos ordenados y no se quiere depender del orden en que entren los valores.

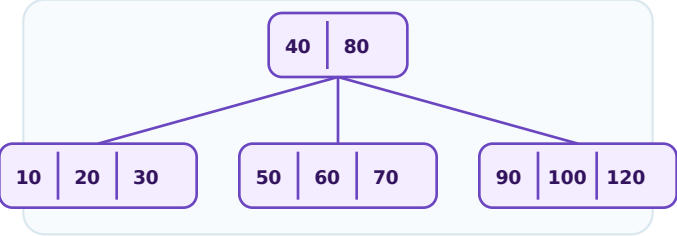
### MAYOR LIMITACIÓN

Tiene mayor complejidad de programación y mantenimiento que un BST simple. En cargas con muchísimas escrituras, las rotaciones pueden agregar costo operativo.

### CASO DE USO EN LA INDUSTRIA

Índices y diccionarios ordenados en memoria para sistemas que necesitan respuesta predecible, por ejemplo catálogos técnicos, motores de reglas o módulos de búsqueda interna con claves numéricas.

## IDENTIFICACIÓN Y VISUALIZACIÓN



Definición: árbol balanceado de múltiples caminos. Un nodo puede almacenar varias claves y varios hijos. En el B+ los datos se concentran normalmente en las hojas, que además pueden estar enlazadas para recorrer rangos.

## ANÁLISIS DE COMPLEJIDAD (BIG O)

| Operación   | Peor caso     | Caso promedio | Observación             |
|-------------|---------------|---------------|-------------------------|
| Búsqueda    | $O(\log_m n)$ | $O(\log_m n)$ | $m$ = ramificación.     |
| Inserción   | $O(\log_m n)$ | $O(\log_m n)$ | Divide nodos llenos.    |
| Eliminación | $O(\log_m n)$ | $O(\log_m n)$ | Fusiona o redistribuye. |

$n$  = registros y  $m$  = orden del árbol. En notación general también se expresa como  $O(\log n)$ .

## CARACTERÍSTICAS MECÁNICAS

Cada nodo guarda varias claves ordenadas y separa rangos entre sus hijos. Cuando un nodo se llena, se divide y una clave sube para mantener la estructura. Cuando un nodo queda con pocas claves, puede redistribuirse con un hermano o fusionarse. Su altura suele ser muy baja, lo que reduce accesos a memoria secundaria.

### MAYOR VENTAJA

Es excelente para bases de datos y sistemas de archivos porque reduce lecturas de disco o SSD. Su alto factor de ramificación permite manejar millones de registros con pocas capas.

### MAYOR LIMITACIÓN

Su implementación es más compleja que la de árboles binarios. Para conjuntos pequeños en memoria puede ser innecesario y generar más sobrecarga que beneficio.

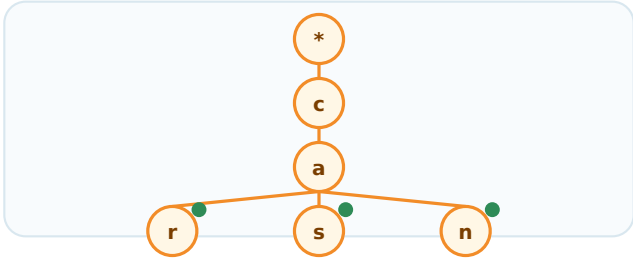
### CASO DE USO EN LA INDUSTRIA

Índices de bases de datos relacionales y sistemas de archivos. Un ejemplo clásico es el uso de árboles B/B+ para localizar registros por clave y soportar búsquedas por rango de forma eficiente.

# Trie (Árbol de Prefijos)

Estructura especializada en cadenas, prefijos y autocompletado

## IDENTIFICACIÓN Y VISUALIZACIÓN



Definición: árbol donde cada nivel representa una parte de la clave, normalmente un carácter. Las palabras se forman siguiendo caminos desde la raíz y los nodos pueden marcar el final de una clave válida.

## ANÁLISIS DE COMPLEJIDAD (BIG O)

| Operación   | Peor caso | Caso promedio | Observación                 |
|-------------|-----------|---------------|-----------------------------|
| Búsqueda    | O(L)      | O(L)          | L = longitud clave.         |
| Inserción   | O(L)      | O(L)          | Crea nodos faltantes.       |
| Eliminación | O(L)      | O(L)          | Quita marca y limpia ramas. |

L = longitud de la palabra o clave. No depende directamente de la cantidad total de palabras.

## CARACTERÍSTICAS MECÁNICAS

Comparte prefijos comunes para evitar repetir caracteres completos. Por ejemplo, las palabras 'car', 'can' y 'casa' comparten los primeros niveles. Para buscar, se recorre carácter por carácter; si falta un enlace, la palabra no existe. Para borrar, se desmarca el final y se eliminan nodos que ya no sirven.

### MAYOR VENTAJA

Es ideal para búsquedas por prefijo, autocompletado, diccionarios y validación rápida de palabras. Puede responder preguntas como: '¿qué palabras empiezan con este texto?'.

### MAYOR LIMITACIÓN

Puede consumir mucha memoria si el alfabeto es grande o las ramas son muy dispersas. Por eso a veces se usan versiones comprimidas como radix tree o Patricia trie.

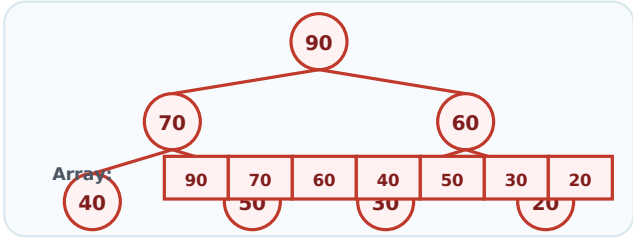
### CASO DE USO EN LA INDUSTRIA

Motores de búsqueda, correctores ortográficos, sugerencias de texto y sistemas de ruteo por prefijos en redes. También se aplica en filtros de palabras y análisis de cadenas.

# Heap (Montículo Min/Max)

Árbol completo para prioridades y extracción rápida del mayor o menor valor

## IDENTIFICACIÓN Y VISUALIZACIÓN



Definición: árbol binario completo que cumple una propiedad de prioridad. En un Max-Heap cada padre es mayor o igual que sus hijos; en un Min-Heap cada padre es menor o igual que sus hijos.

## ANÁLISIS DE COMPLEJIDAD (BIG O)

| Operación   | Peor caso   | Caso promedio | Observación         |
|-------------|-------------|---------------|---------------------|
| Búsqueda    | $O(n)$      | $O(n)$        | Valor arbitrario.   |
| Inserción   | $O(\log n)$ | $O(\log n)$   | Sube por prioridad. |
| Eliminación | $O(\log n)$ | $O(\log n)$   | Extracción raíz.    |

$n$  = cantidad de elementos. Consultar la raíz es  $O(1)$ , pero buscar cualquier valor no es eficiente.

## CARACTERÍSTICAS MECÁNICAS

Se representa casi siempre con un arreglo, porque al ser completo no necesita punteros explícitos. Para un índice  $i$ , sus hijos suelen ubicarse en  $2i+1$  y  $2i+2$ . Al insertar se coloca al final y sube si rompe la prioridad. Al extraer la raíz, el último elemento baja hasta restaurar la propiedad del heap.

### MAYOR VENTAJA

Es muy eficiente para colas de prioridad: permite obtener rápidamente el elemento más urgente, más pequeño o más grande, según el tipo de heap utilizado.

### MAYOR LIMITACIÓN

No mantiene todos los elementos ordenados como un BST. Si se desea buscar un valor específico o recorrer en orden, el heap no es la mejor estructura.

### CASO DE USO EN LA INDUSTRIA

Colas de prioridad en sistemas operativos, planificación de tareas y algoritmos como Dijkstra o A\*. En Java, la clase PriorityQueue se basa en una estructura de montículo.